

Doc. No.:	P0941R0
Date:	2018-05-04
Reply to:	Ville Voutilainen
Audience:	EWG, LEWG

Integrating feature-test macros into the C++ WD

[-1] Preface

This revision of this paper proposes incorporating feature-testing macros into the working draft of the C++ standard.

[0] Contents

1. [Introduction](#)
2. [Wording](#)
 1. [Testing for the presence of an attribute: `\_\_has\_cpp\_attribute`](#)
 2. [Feature-testing macros](#)
3. [Feedback question on RTTI and exceptions](#)
4. [Revision history](#)

[1] Introduction

At the November 2017 (Albuquerque) meeting of WG21, there was a five-way poll of all of the C++ experts in attendance concerning their support for "Feature macros should be standardized in the IS, and we want to invite a proposal to be reviewed by EWG to discuss the rationale and granularity." The results of the poll:

Strongly favor	Favor	Neutral	Oppose	Strongly oppose
27	21	9	4	1

The approach taken here is as follows:

- For the macros incorporated, use the latest value from SD-6, and do not bump the numbers. This is done in order to avoid pointless breakage of existing code that uses the SD-6 macros.
- Library macros are defined both in the corresponding header for the library facility and in `<version>`.

[2] Wording

[2.1] Testing for the presence of an attribute: `__has_cpp_attribute`

After [lex.literal], insert a new subsection of section [lex] with the following content:

has-attribute-expression:
`__has_cpp_attribute ( attribute-token )`

A *has-attribute-expression* shall appear only in the controlling constant expression of a `#if` or `#elif` directive ([cpp.cond] 16.1). The *has-attribute-expression* is replaced by a non-zero pp-number if the

implementation supports an attribute with the specified name, and by the pp-number 0 otherwise.

For a standard attribute, the value of the `__has_cpp_attribute` macro is based on the year and month in which the attribute was voted into the working draft. In the case where the attribute is vendor-specific, the value is implementation-defined. However, in most cases it is expected that the availability of an attribute can be detected by any non-zero result.

The `#ifdef` and `#ifndef` directives, and the `defined` conditional inclusion operator, shall treat `__has_cpp_attribute` as if it were the name of a defined macro. The identifier `__has_cpp_attribute` shall not appear in any context not mentioned in this section.

This demonstrates a way to use the attribute `[[deprecated]]` only if it is available.

```
#ifndef __has_cpp_attribute
# define __has_cpp_attribute(x) 0
#endif
#if __has_cpp_attribute(deprecated)
# define ATTR_DEPRECATED(msg) [[deprecated(msg)]]
#else
# define ATTR_DEPRECATED(msg)
#endif
```

[2.2] Feature-testing macros

Drafting note: these are intended to match the latest values in SD-6. Wording review should check that that is indeed so.

After [support.runtime], insert a new subsection of section [language.support] with the following content:

The macros that are not predefined are defined in the header `<version>` in addition to the header specified in the table.

Macro name	Value	Header
<code>__cpp_unicode_characters</code>	200704	<i>predefined</i>
<code>__cpp_raw_strings</code>	200710	<i>predefined</i>
<code>__cpp_unicode_literals</code>	200710	<i>predefined</i>
<code>__cpp_user_defined_literals</code>	200809	<i>predefined</i>
<code>__cpp_threadsafe_static_init</code>	200806	<i>predefined</i>
<code>__cpp_lambdas</code>	200907	<i>predefined</i>
<code>__cpp_range_based_for</code>	200907	<i>predefined</i>
<code>__cpp_static_assert</code>	200410	<i>predefined</i>
<code>__cpp_decltype</code>	200707	<i>predefined</i>
<code>__cpp_attributes</code>	200809	<i>predefined</i>
<code>__has_cpp_attribute(noreturn)</code>	200809	<i>predefined</i>
<code>__has_cpp_attribute(carries_dependency)</code>	200809	<i>predefined</i>
<code>__cpp_rvalue_references</code>	200610	<i>predefined</i>
<code>__cpp_variadic_templates</code>	200704	<i>predefined</i>
<code>__cpp_initializer_lists</code>	200806	<i>predefined</i>
<code>__cpp_delegating_constructors</code>	200604	<i>predefined</i>

Macro name	Value	Header
<code>_cpp_nsdmi</code>	200809	<i>predefined</i>
<code>_cpp_ref_qualifiers</code>	200710	<i>predefined</i>
<code>_cpp_alias_templates</code>	200704	<i>predefined</i>
<code>_cpp_binary_literals</code>	201304	<i>predefined</i>
<code>_cpp_init_captures</code>	201304	<i>predefined</i>
<code>_cpp_generic_lambdas</code>	201304	<i>predefined</i>
<code>_cpp_sized_deallocation</code>	201309	<i>predefined</i>
<code>_cpp_decltype_auto</code>	201304	<i>predefined</i>
<code>_cpp_return_type_deduction</code>	201304	<i>predefined</i>
<code>_has_cpp_attribute(deprecated)</code>	201309	<i>predefined</i>
<code>_cpp_aggregate_nsdmi</code>	201304	<i>predefined</i>
<code>_cpp_variable_templates</code>	201304	<i>predefined</i>
<code>_cpp_lib_integer_sequence</code>	201304	<utility>
<code>_cpp_lib_exchange_function</code>	201304	<utility>
<code>_cpp_lib_tuples_by_type</code>	201304	<utility>
<code>_cpp_lib_tuple_element_t</code>	201402	<utility>
<code>_cpp_lib_make_unique</code>	201304	<memory>
<code>_cpp_lib_transparent_operators</code>	201210	<functional>
<code>_cpp_lib_integral_constant_callable</code>	201304	<type_traits>
<code>_cpp_lib_transformation_trait_aliases</code>	201304	<type_traits>
<code>_cpp_lib_result_of_sfinae</code>	201210	<functional> <type_traits>
<code>_cpp_lib_is_final</code>	201402	<type_traits>
<code>_cpp_lib_is_null_pointer</code>	201309	<type_traits>
<code>_cpp_lib_string_udls</code>	201304	<string>
<code>_cpp_lib_generic_associative_lookup</code>	201304	<map> <set>
<code>_cpp_lib_null_iterators</code>	201304	<iterator>
<code>_cpp_lib_make_reverse_iterator</code>	201402	<iterator>
<code>_cpp_lib_robust_nonmodifying_seq_ops</code>	201304	<algorithm>
<code>_cpp_lib_complex_udls</code>	201309	<complex>
<code>_cpp_lib_quoted_string_io</code>	201304	<iomanip>
<code>_has_include(<shared_mutex>)</code>	1	<i>predefined</i>
<code>_cpp_lib_shared_timed_mutex</code>	201402	<shared_mutex>
<code>_cpp_hex_float</code>	201603	<i>predefined</i>
<code>_cpp_inline_variables</code>	201606	<i>predefined</i>
<code>_cpp_aligned_new</code>	201606	<i>predefined</i>
<code>_cpp_guaranteed_copy_elision</code>	201606	<i>predefined</i>
<code>_cpp_noexcept_function_type</code>	201510	<i>predefined</i>
<code>_cpp_fold_expressions</code>	201411	<i>predefined</i>

Macro name	Value	Header
<code>_cpp_capture_star_this</code>	201603	<i>predefined</i>
<code>_cpp_constexpr</code>	201603	<i>predefined</i>
<code>_cpp_if_constexpr</code>	201606	<i>predefined</i>
<code>_cpp_range_based_for</code>	201603	<i>predefined</i>
<code>_cpp_static_assert</code>	201411	<i>predefined</i>
<code>_cpp_deduction_guides</code>	201606	<i>predefined</i>
<code>_cpp_deduction_guides</code>	201611	<i>predefined</i>
<code>_cpp_nontype_template_parameter_auto</code>	201606	<i>predefined</i>
<code>_cpp_namespace_attributes</code>	201411	<i>predefined</i>
<code>_cpp_enumerator_attributes</code>	201411	<i>predefined</i>
<code>_cpp_inheriting_constructors</code>	201511	<i>predefined</i>
<code>_cpp_variadic_using</code>	201611	<i>predefined</i>
<code>_has_cpp_attribute(fallthrough)</code>	201603	<i>predefined</i>
<code>_has_cpp_attribute(nodiscard)</code>	201603	<i>predefined</i>
<code>_has_cpp_attribute(maybe_unused)</code>	201603	<i>predefined</i>
<code>_cpp_structured_bindings</code>	201606	<i>predefined</i>
<code>_cpp_aggregate_bases</code>	201603	<i>predefined</i>
<code>_cpp_nontype_template_args</code>	201411	<i>predefined</i>
<code>_cpp_template_template_args</code>	201611	<i>predefined</i>
<code>_cpp_fold_expressions</code>	201603	<i>predefined</i>
<code>_has_include</code>	<i>defined</i>	<i>predefined</i>
<code>_cpp_lib_byte</code>	201603	<cstdint>
<code>_cpp_lib_hardware_interference_size</code>	201703	<new>
<code>_cpp_lib_launder</code>	201606	<new>
<code>_cpp_lib_uncaught_exceptions</code>	201411	<exception>
<code>_cpp_lib_as_const</code>	201510	<utility>
<code>_cpp_lib_make_from_tuple</code>	201606	<tuple>
<code>_cpp_lib_apply</code>	201603	<tuple>
<code>_has_include(<optional>)</code>	1	<i>predefined</i>
<code>_cpp_lib_optional</code>	201603	<optional>
<code>_has_include(<any>)</code>	1	<i>predefined</i>
<code>_cpp_lib_any</code>	201603	<any>
<code>_has_include(<memory_resource>)</code>	1	<i>predefined</i>
<code>_cpp_lib_memory_resource</code>	201603	<memory_resource>
<code>_cpp_lib_boyer_moore_searcher</code>	201603	<functional>
<code>_has_include(<string_view>)</code>	1	<i>predefined</i>
<code>_cpp_lib_string_view</code>	201603	<string_view>
<code>_cpp_lib_sample</code>	201603	<algorithm>
<code>_cpp_lib_any</code>	201606	<any>
<code>_cpp_lib_optional</code>	201606	<optional>

Macro name	Value	Header
__cpp_lib_optional	201606	<optional>
__has_include(<variant>)	1	<i>predefined</i>
__cpp_lib_variant	201606	<variant>
__cpp_lib_addressof_constexpr	201603	<memory>
__cpp_lib_raw_memory_algorithms	201606	<memory>
__cpp_lib_transparent_operators	201510	<memory> <functional>
__cpp_lib_enable_shared_from_this	201603	<memory>
__cpp_lib_shared_ptr_weak_type	201606	<memory>
__cpp_lib_shared_ptr_arrays	201611	<memory>
__cpp_lib_invoke	201411	<functional>
__cpp_lib_not_fn	201603	<functional>
__cpp_lib_void_t	201411	<type_traits>
__cpp_lib_bool_constant	201505	<type_traits>
__cpp_lib_type_trait_variable_templates	201510	<type_traits>
__cpp_lib_logical_traits	201510	<type_traits>
__cpp_lib_is_swappable	201603	<type_traits>
__cpp_lib_is_invocable	201703	<type_traits>
__cpp_lib_has_unique_object_representations	201606	<type_traits>
__cpp_lib_is_aggregate	201703	<type_traits>
__cpp_lib_chrono	201611	<chrono>
__has_include(<execution>)	1	<i>predefined</i>
__cpp_lib_execution	201603	<execution>
__cpp_lib_parallel_algorithm	201603	<algorithm> <numeric>
__cpp_lib_to_chars	201611	<utility>
__cpp_lib_string_view	201606	<string> <string_view>
__cpp_lib_allocator_traits_is_always_equal	201411	<memory> <scoped_allocator> <string> <deque> <forward_list> <list> <vector> <map> <set> <unordered_map> <unordered_set>
__cpp_lib_incomplete_container_elements	201505	<forwardlist> <list> <vector>
__cpp_lib_map_try_emplace	201411	<map>
__cpp_lib_unordered_map_try_emplace	201411	<unordered_map>
__cpp_lib_node_extract	201606	<map> <set> <unordered_map> <unordered_set>
__cpp_lib_array_constexpr	201603	<iterator> <array>

Macro name	Value	Header
__cpp_lib_nonmember_container_access	201411	<iterator> <array> <deque> <forward_list> <list> <map> <regex> <set> <string> <unordered_map> <unordered_set> <vector>
__cpp_lib_clamp	201603	<algorithm>
__cpp_lib_gcd_lcm	201606	<numeric>
__cpp_lib_hypot	201603	<cmath>
__cpp_lib_math_special_functions	201603	<cmath>
__has_include(<filesystem>)	1	<i>predefined</i>
__cpp_lib_filesystem	201703	<filesystem>
__cpp_lib_atomic_is_always_lock_free	201603	<atomic>
__cpp_lib_shared_mutex	201505	<shared_mutex>
__cpp_lib_scoped_lock	201703	<mutex>

[3] Feedback question on RTTI and exceptions

With very few exceptions, the features of C++98 have all been implemented in virtually every C++ compiler. But in many compilers, some of them can be enabled/disabled. It is recommended that the macros in the following table should be used to indicate whether one of these features is enabled in the current compilation.

Macro name	Value	Header
__cpp_rtti	199711	<i>predefined</i>
__cpp_exceptions	199711	<i>predefined</i>

[4] Revision history

Date	Document	Description
	P0941R0	Initial draft
	P0941R1	Draft wording, with non-standard content removed and rtti/exception macros made a separate point looking for feedback.